Guía Práctica

Business Process Execution Language

BPEL con Spring Boot + Spring-WS

Asignatura	Aplicaciones con Enfoque Orientado a Servicios
Sesión N°	6
Herramientas	IntelliJ IDEA Community · Spring Boot · Spring-WS
Modalidad	Práctica guiada + Actividad grupal

Contenido de la Guía

PARTE A Ejemplo Guiado por el Docente

A.1 ¿Qué vamos a construir?

A.2 Creación del proyecto Spring Boot

A.3 Servicio Web SOAP: Saludar

A.4 Servicio Web SOAP: CalcularEdad

A.5 Diagrama BPMN del proceso

A.6 Mapeo BPMN → BPEL (código .bpel completo)

A.7 Prueba con SoapUI / Postman

PARTE B Actividad Grupal (estudiantes)

B.1 Enunciado del caso: Tienda Online

B.2 Entregables y rúbrica

SOLUCIONARIO Solución completa – Parte B

PARTE A — Ejemplo Guiado por el Docente

■ Objetivo del ejemplo

Construir dos servicios web SOAP con Spring Boot y orquestarlos mediante un proceso BPEL secuencial, siguiendo el ciclo: **BPMN** → **mapeo** → **código .bpel**.

Al finalizar el ejemplo el estudiante podrá replicar el mismo patrón para el caso práctico grupal.

A.1 ¿Qué vamos a construir?

Construiremos dos servicios web SOAP independientes:

Saludar	Recibe: nombre (String)	Retorna: "Hola " + nombre
CalcularEdad	Recibe: anioNacimiento (int)	Retorna: edad aproximada (int)

Luego crearemos un archivo **.bpel** que orquesta ambos servicios de forma secuencial.

A.2 Creación del proyecto Spring Boot en IntelliJ

■ Requisitos previos

- IntelliJ IDEA Community (cualquier versión reciente)
- JDK 17 o superior instalado
- Conexión a internet (para descargar dependencias Maven)

Paso 1 — Crear proyecto

Ve a **File** → **New** → **Project**. Selecciona **Spring Initializr**.

Si no aparece Spring Initializr en IntelliJ Community, elige **Maven** y luego edita el pom.xml a mano (se muestra el pom completo en el paso siguiente).

Paso 2 **Configurar metadatos**

Completa los campos:

- **Group:** com.idat
- **Artifact:** bpel-demo
- **Name:** bpel-demo
- **Packaging:** Jar
- **Java:** 17

Paso 3 **Agregar dependencias**

Selecciona: **Spring Web Services** (spring-ws-core) y **Spring Web**.

Si usas el pom manual, copia exactamente el XML del paso A.3.

pom.xml completo (copia esto si creaste el proyecto como Maven vacío):

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>

  <groupId>com.idat</groupId>
  <artifactId>bpel-demo</artifactId>
```

```
    <version>0.0.1-SNAPSHOT</version>
    <name>bpel-demo</name>

    <properties>
      <java.version>17</java.version>
    </properties>

    <dependencies>
      <!-- Spring Boot Web -->
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
      </dependency>

      <!-- Spring Web Services (SOAP) -->
      <dependency>
        <groupId>org.springframework.ws</groupId>
        <artifactId>spring-ws-core</artifactId>
      </dependency>

      <!-- WSDL generator desde XSD -->
      <dependency>
        <groupId>wsdl4j</groupId>
        <artifactId>wsdl4j</artifactId>
      </dependency>
    </dependencies>

    <build>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
```

```
      </plugin>
    </plugins>
  </build>
</project>
```

A.3 Servicio Web SOAP: Saludar

Estructura de carpetas que crearemos:

```
src/main/java/com/idat/bpeldemo/
  BpelDemoApplication.java          ← clase principal (ya existe)
  config/
    WebServiceConfig.java            ← configuración Spring-WS
  endpoint/
    SaludarEndpoint.java             ← lógica del servicio Saludar

src/main/resources/
  saludar.xsd                        ← schema XML del servicio
```

Archivo: src/main/resources/saludar.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:tns="http://idat.com/saludar"
           targetNamespace="http://idat.com/saludar"
           elementFormDefault="qualified">

  <!-- Mensaje de petición: recibe un nombre -->
  <xs:element name="SaludarRequest">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="nombre" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
```

```
  </xs:element>

  <!-- Mensaje de respuesta: retorna el saludo -->
  <xs:element name="SaludarResponse">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="saludo" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

Archivo: SaludarEndpoint.java

```
package com.idat.bpeldemo.endpoint;

import org.springframework.ws.server.endpoint.annotation.Endpoint;
import org.springframework.ws.server.endpoint.annotation.PayloadRoot;
import org.springframework.ws.server.endpoint.annotation.RequestPayload;
import org.springframework.ws.server.endpoint.annotation.ResponsePayload;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import javax.xml.parsers.DocumentBuilderFactory;

@Endpoint
public class SaludarEndpoint {

    private static final String NS = "http://idat.com/saludar";

    @PayloadRoot(namespace = NS, localPart = "SaludarRequest")
    @ResponsePayload
    public Element saludar(@RequestPayload Element request) throws Exception {
        // 1. Leer el nombre del request XML
```

```
        String nombre = request
            .getElementsByTagNameNS(NS, "nombre")
            .item(0).getTextContent();

        // 2. Construir la respuesta XML
        Document doc = DocumentBuilderFactory
            .newInstance().newDocumentBuilder().newDocument();

        Element response = doc.createElementNS(NS, "SaludarResponse");
        Element saludo   = doc.createElementNS(NS, "saludo");
        saludo.setTextContent("Hola " + nombre);
        response.appendChild(saludo);

        return response;
    }
}
```

Archivo: WebServiceConfig.java — configura el endpoint y publica el WSDL automáticamente

```
package com.idat.bpeldemo.config;

import org.springframework.boot.web.servlet.ServletRegistrationBean;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.ws.config.annotation.EnableWs;
import org.springframework.ws.config.annotation.WsConfigurerAdapter;
import org.springframework.ws.transport.http.MessageDispatcherServlet;
import org.springframework.ws.wsdl.wsdl11.DefaultWsdl11Definition;
import org.springframework.xml.xsd.SimpleXsdSchema;
import org.springframework.xml.xsd.XsdSchema;

@EnableWs
```

```
@Configuration
public class WebServiceConfig extends WsConfigurerAdapter {

    // ■■ Registrar el servlet Spring-WS en /ws/* ■■■■■■■■■■■■■■■■■■■■■■
    @Bean
    public ServletRegistrationBean<MessageDispatcherServlet> servlet(
            ApplicationContext ctx) {
        MessageDispatcherServlet servlet = new MessageDispatcherServlet();
        servlet.setApplicationContext(ctx);
        servlet.setTransformWsdlLocations(true);
        return new ServletRegistrationBean<>(servlet, "/ws/*");
    }

    // ■■ WSDL para Saludar ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    @Bean(name = "saludar")
    public DefaultWsdl11Definition saludarWsdl(XsdSchema saludarSchema) {
        DefaultWsdl11Definition def = new DefaultWsdl11Definition();
        def.setPortTypeName("SaludarPort");
        def.setLocationUri("/ws");
        def.setTargetNamespace("http://idat.com/saludar");
        def.setSchema(saludarSchema);
        return def;
    }

    @Bean
    public XsdSchema saludarSchema() {
        return new SimpleXsdSchema(new ClassPathResource("saludar.xsd"));
    }

    // ■■ WSDL para CalcularEdad ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    @Bean(name = "calcularEdad")
    public DefaultWsdl11Definition calcularEdadWsdl(XsdSchema calcularEdadSchema) {
        DefaultWsdl11Definition def = new DefaultWsdl11Definition();
        def.setPortTypeName("CalcularEdadPort");
```

```
        def.setLocationUri("/ws");
        def.setTargetNamespace("http://idat.com/calcularedad");
        def.setSchema(calcularEdadSchema);
        return def;
    }

    @Bean
    public XsdSchema calcularEdadSchema() {
        return new SimpleXsdSchema(new ClassPathResource("calcularEdad.xsd"));
    }
}
```

A.4 Servicio Web SOAP: CalcularEdad

Archivo: src/main/resources/calcularEdad.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:tns="http://idat.com/calcularedad"
           targetNamespace="http://idat.com/calcularedad"
           elementFormDefault="qualified">

  <xs:element name="CalcularEdadRequest">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="anioNacimiento" type="xs:int"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

  <xs:element name="CalcularEdadResponse">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="edad" type="xs:int"/>
```

```
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

Archivo: CalcularEdadEndpoint.java

```
package com.idat.bpeldemo.endpoint;

import org.springframework.ws.server.endpoint.annotation.Endpoint;
import org.springframework.ws.server.endpoint.annotation.PayloadRoot;
import org.springframework.ws.server.endpoint.annotation.RequestPayload;
import org.springframework.ws.server.endpoint.annotation.ResponsePayload;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import javax.xml.parsers.DocumentBuilderFactory;
import java.time.Year;

@Endpoint
public class CalcularEdadEndpoint {

    private static final String NS = "http://idat.com/calcularedad";

    @PayloadRoot(namespace = NS, localPart = "CalcularEdadRequest")
    @ResponsePayload
    public Element calcularEdad(@RequestPayload Element request) throws Exception {
        // 1. Leer anioNacimiento
        int anio = Integer.parseInt(request
            .getElementsByTagNameNS(NS, "anioNacimiento")
            .item(0).getTextContent());

        // 2. Calcular edad aproximada
        int edad = Year.now().getValue() - anio;
```

```
        // 3. Construir respuesta XML
        Document doc = DocumentBuilderFactory
            .newInstance().newDocumentBuilder().newDocument();

        Element response = doc.createElementNS(NS, "CalcularEdadResponse");
        Element edadEl   = doc.createElementNS(NS, "edad");
        edadEl.setTextContent(String.valueOf(edad));
        response.appendChild(edadEl);

        return response;
    }
}
```

■ ■ Ejecutar el proyecto

1. Clic derecho en **BpelDemoApplication.java** → **Run**

2. Verifica en el navegador:

```
http://localhost:8080/ws/saludar.wsdl

http://localhost:8080/ws/calcularEdad.wsdl
```

Si ves el XML del WSDL, ¡los servicios están activos!

A.5 Diagrama BPMN del proceso (para dibujar en draw.io)

Dibuja este flujo en **draw.io** usando formas BPMN:

Elemento BPMN	Símbolo	Descripción en nuestro proceso
Evento de Inicio	■ (verde)	El cliente envía nombre y año de nacimiento
Tarea de Servicio	■ con engrane	Invocar Saludar(nombre)
Tarea de Servicio	■ con engrane	Invocar CalcularEdad(anioNacimiento)
Tarea de Servicio	■ con engrane	Guardar resultado en variable

Evento de Fin	■ (rojo)	Retornar respuesta combinada al cliente

Flujo secuencial: Inicio → Saludar → CalcularEdad → Guardar → Fin

A.6 Mapeo BPMN → BPEL y archivo .bpel completo

Elemento BPMN	Etiqueta BPEL	Función
Evento de Inicio	`<receive>`	Recibe el mensaje del cliente
Tarea Saludar	`<invoke>`	Llama al WS Saludar
Tarea CalcularEdad	`<invoke>`	Llama al WS CalcularEdad
Tarea Guardar	`<assign>`	Copia resultado a variable
Evento de Fin	`<reply>`	Responde al cliente
Flujo secuencial	`<sequence>`	Envuelve todas las actividades en orden

Archivo: proceso-saludar-edad.bpel — proceso BPEL completo

```
<?xml version="1.0" encoding="UTF-8"?>

<process name="SaludarYCalcularEdad"

         targetNamespace="http://idat.com/proceso"

         xmlns="http://docs.oasis-open.org/wsbpel/2.0/process/executable"

         xmlns:sal="http://idat.com/saludar"

         xmlns:edad="http://idat.com/calcularedad"

         xmlns:cliente="http://idat.com/cliente">


  <!-- ■■■■■■■■■ PARTNER LINKS ■■■■■■■■■ -->

  <partnerLinks>


    <!-- El cliente que invoca este proceso -->

    <partnerLink name="cliente"

                 partnerLinkType="cliente:ClienteLT"

                 myRole="SaludarEdadProvider"/>


    <!-- Servicio externo Saludar -->

    <partnerLink name="saludarService"

                 partnerLinkType="sal:SaludarLT"
```

```
                partnerRole="SaludarProvider"/>

    <!-- Servicio externo CalcularEdad -->
    <partnerLink name="calcularEdadService"
                partnerLinkType="edad:CalcularEdadLT"
                partnerRole="CalcularEdadProvider"/>
  </partnerLinks>

  <!-- ■■■■■■■■■ VARIABLES ■■■■■■■■■ -->
  <variables>
    <!-- Datos de entrada del cliente -->
    <variable name="inputRequest"  messageType="cliente:SaludarEdadRequest"/>
    <variable name="outputResponse" messageType="cliente:SaludarEdadResponse"/>

    <!-- Mensajes para el servicio Saludar -->
    <variable name="saludarRequest"  messageType="sal:SaludarRequestMessage"/>
    <variable name="saludarResponse" messageType="sal:SaludarResponseMessage"/>

    <!-- Mensajes para el servicio CalcularEdad -->
    <variable name="edadRequest"  messageType="edad:CalcularEdadRequestMessage"/>
    <variable name="edadResponse" messageType="edad:CalcularEdadResponseMessage"/>
  </variables>

  <!-- ■■■■■■■■■ CUERPO PRINCIPAL (sequence = flujo BPMN) ■■■■■■■■■ -->
  <sequence>

    <!-- EVENTO INICIO: recibir petición del cliente -->
    <receive name="RecibirPeticion"
             partnerLink="cliente"
             portType="cliente:SaludarEdadPT"
             operation="procesarSaludarEdad"
             variable="inputRequest"
             createInstance="yes"/>

    <!-- TAREA 1: preparar mensaje para Saludar -->
```

```
<assign name="PrepararSaludar">
  <copy>
    <from>$inputRequest.payload/nombre</from>
    <to>$saludarRequest.payload/nombre</to>
  </copy>
</assign>

<!-- TAREA 2 (invoke): llamar al WS Saludar -->
<invoke name="InvocarSaludar"
        partnerLink="saludarService"
        portType="sal:SaludarPort"
        operation="saludar"
        inputVariable="saludarRequest"
        outputVariable="saludarResponse"/>

<!-- TAREA 3: preparar mensaje para CalcularEdad -->
<assign name="PrepararCalcularEdad">
  <copy>
    <from>$inputRequest.payload/anioNacimiento</from>
    <to>$edadRequest.payload/anioNacimiento</to>
  </copy>
</assign>

<!-- TAREA 4 (invoke): llamar al WS CalcularEdad -->
<invoke name="InvocarCalcularEdad"
        partnerLink="calcularEdadService"
        portType="edad:CalcularEdadPort"
        operation="calcularEdad"
        inputVariable="edadRequest"
        outputVariable="edadResponse"/>

<!-- TAREA 5 (assign): guardar resultados en la respuesta -->
<assign name="GuardarResultados">
  <copy>
```

```
        <from>$saludarResponse.payload/saludo</from>
        <to>$outputResponse.payload/saludo</to>
      </copy>
      <copy>
        <from>$edadResponse.payload/edad</from>
        <to>$outputResponse.payload/edad</to>
      </copy>
    </assign>

    <!-- EVENTO FIN: responder al cliente -->
    <reply name="ResponderCliente"
           partnerLink="cliente"
           portType="cliente:SaludarEdadPT"
           operation="procesarSaludarEdad"
           variable="outputResponse"/>

  </sequence>
</process>
```

A.7 Probar los servicios con SoapUI

Para probar el servicio **Saludar**, envía este SOAP Request:

```
POST http://localhost:8080/ws
Content-Type: text/xml

<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:sal="http://idat.com/saludar">
  <soapenv:Header/>
  <soapenv:Body>
    <sal:SaludarRequest>
      <sal:nombre>Maria</sal:nombre>
```

```
      </sal:SaludarRequest>
    </soapenv:Body>
  </soapenv:Envelope>
```

Respuesta esperada:

```
<soapenv:Envelope ...>
  <soapenv:Body>
    <sal:SaludarResponse>
      <sal:saludo>Hola Maria</sal:saludo>
    </sal:SaludarResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

Para probar el servicio **CalcularEdad**:

```
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ed="http://idat.com/calcularedad">
  <soapenv:Header/>
  <soapenv:Body>
    <ed:CalcularEdadRequest>
      <ed:anioNacimiento>1998</ed:anioNacimiento>
    </ed:CalcularEdadRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

PARTE B — Actividad Grupal (Estudiantes)

■ Instrucciones generales

• Grupos de **3 a 4 integrantes**.

• Tiempo de trabajo: **30 minutos**.

• Al terminar, **un integrante sube** los archivos al aula virtual.

• Se expone brevemente (2 min por grupo) al finalizar.

B.1 Caso Práctico: Sistema de Tienda Online

Una tienda online desea automatizar su proceso de venta. Al recibir un pedido de un cliente, el sistema debe ejecutar tres operaciones de forma secuencial, cada una implementada como un servicio web SOAP independiente:

WS 1: VerificarStock	Recibe: codigoProducto (String)	Retorna: hayStock (boolean) — true si hay stock, false si no
WS 2: CalcularTotal	Recibe: precio (double), cantidad (int)	Retorna: total (double) — precio × cantidad
WS 3: GenerarOrden	Recibe: cliente (String), total (double)	Retorna: numeroOrden (String) — ej: 'ORD-2024-' + cliente

Flujo BPMN del proceso:

Inicio → VerificarStock → CalcularTotal → GenerarOrden → Fin

B.2 Entregables y Rúbrica

#	Entregable	Descripción	Puntos
1	XSD (3 archivos)	Un archivo .xsd por cada servicio web	15

2	Endpoints Java (3)	Una clase @Endpoint por servicio	20
3	WebServiceConfig.java	Configuración que publica los 3 WSDL	15
4	Diagrama BPMN	Captura del diagrama en draw.io o Lucidchart	20
5	Tabla de mapeo	Tabla BPMN → BPEL en Word o PDF	10
6	Archivo .bpel	Proceso BPEL completo con los 3 servicios	20
		TOTAL	**100**